More practice - Topological sort and cycles

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
MEDIUM

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

More practice - Topological sort and cycles | 1 questions | tap a title to jump to it

M 1. Shortest Directed Cycle Through a Node **1Point3Acres no date**

STUDY PATH: question 126 of 167 (more practice)

Priority score 1.7 | Topological sort and cycles | no date | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 1 of 1

Shortest Directed Cycle Through a Node

STUDY: TALK | **ALL TIME 5** | **30 DAYS 0** | **PRIORITY 174/228**

ABOUT THIS QUESTION

Topic: More practice - Topological sort and cycles

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2608 Shortest Cycle in a Graph (related)

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/shortest-cycle-through-node

QUESTION

EXAMPLES

```
Edges [(0,1),(1,2),(2,0)], v = 0 -> 3.
Edges [(0,1),(1,2)], v = 0 -> -1.
```

APPROACH

Write the directed BFS-from-source version from memory in under 12 min.
Practice the "re-entry detection" pattern (don't mark v visited until after pop).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether edges are directed (yes), whether self loops count (cycle of length 1), what to return if there is none (-1), and whether edges have weights.

BRUTE FORCE TO SAY OUT LOUD

Enumerate all simple paths from v back to v with DFS. Exponential.

OPTIMAL IDEA

BFS from v. The first time you see an edge from a visited node u back to v, the cycle length is dist[u] + 1, and BFS order makes it the shortest. Do not mark v as discovered before checking for re-entry. O(V + E).

DRY-RUN THESE CASES

A triangle (3). A path with no return (-1). A self loop (1). Two cycles through v of different lengths.

LIKELY FOLLOW-UPS

Shortest cycle in the whole graph (BFS from every node), weighted edges (Dijkstra), or undirected graphs.

PITFALLS

Marking v visited at the start, which hides the cycle. Counting nodes instead of edges.

HOW TO THINK ABOUT IT

A cycle through v is a path from v back to v, so find shortest distances from v and watch for an edge that points back at v. Breadth-first search discovers nodes in order of distance, so the first edge back to v closes the shortest cycle. Do not mark v as 'seen' in a way that hides the edge that returns to it.

Basis: Related to LeetCode 2608, Shortest Cycle in a Graph, but directed and anchored at a given node.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = INF
def dfs(u, length, on_path):
    global best
    for w in graph[u]:
        if w == v:
            best = min(best, length + 1)
        elif w not in on_path:
            dfs(w, length + 1, on_path | {w})
dfs(v, 0, {v})
answer = -1 if best == INF else best
```

TIME	SPACE
exponential	O(V)

WHY IT IS SLOW

The number of simple paths can be exponential in the graph size. A breadth-first search from v reaches the shortest cycle in O(V + E) because it visits nodes in order of distance.

OPTIMAL SOLUTION · BFS from v, close the cycle on the first edge back to v

TIME	SPACE
O(V + E)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you find the shortest cycle in the whole graph?

Run the same BFS from every vertex and take the minimum: O(V * (V + E)). For undirected graphs, BFS from each node and treat any non-tree edge as closing a cycle of length dist[u] + dist[w] + 1 (the LeetCode 2608 version).

FOLLOW-UP 2 What if edges have positive weights?

Replace BFS with Dijkstra from v and close the cycle at the cheapest dist[u] + weight(u, v). That is O((V + E) log V) with a heap. With negative weights use Bellman-Ford and handle negative cycles separately.

FOLLOW-UP 3 The shortest cycle anywhere in the graph (the girth).

Run the same BFS from every node and take the smallest result: O(V * (V + E)). That is the standard way and fine for graphs up to a few thousand nodes.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Shortest Directed Cycle Through a Node
#
# Examples: Edges [(0,1),(1,2),(2,0)], v = 0 -> 3. Edges
# [(0,1),(1,2)], v = 0 -> -1.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you find the shortest cycle in the whole
# graph?
# 2. What if edges have positive weights?
# 3. The shortest cycle anywhere in the graph (the girth).
#
# INTERVIEW NOTES
# A cycle through v is a path from v back to v, so find
# shortest distances from v and watch for an edge that
# points back at v. Breadth-first search discovers nodes
# in order of distance, so the first edge back to v closes
# the shortest cycle. Do not mark v as 'seen' in a way
# that hides the edge that returns to it.
#
# BRUTE FORCE (say it first): Enumerate every simple cycle
# through v with DFS
# Explore all simple paths from v and record the length
# whenever an edge returns to v. Exact and the oracle, but
# the number of simple paths can be exponential. Time
# exponential. Space O(V).


import heapq
from collections import deque


# BRUTE FORCE (say it first): Enumerate every simple cycle
# through v with DFS | time exponential
def shortest_cycle_dfs(n, edges, v):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
    best = [float("inf")]

    def go(u, seen, length):
        for w in adj[u]:
            if w == v:
                best[0] = min(best[0], length + 1)
            elif w not in seen:
                go(w, seen | {w}, length + 1)
```

```
    go(v, {v}, 0)
    return -1 if best[0] == float("inf") else best[0]


# HELPERS for the base question
def make_graph(n, edges):
    adj = [[] for _ in range(n)]
    for a, b, *w in edges:
        # edges may carry a weight (default 1)
        adj[a].append((b, w[0] if w else 1))
    return adj


# HELPERS for follow-up 2
INF = float("inf")


# BASE QUESTION (optimal): BFS from v, close the cycle on
# the first edge back to v
# Idea: A directed cycle through v is a path v to u
# followed by an edge u to v. BFS visits nodes in
# nondecreasing distance, so the first time we see an edge
# back to v its length is dist[u] + 1 and is the shortest.
def shortest_cycle_through(n, edges, v):
    # BFS from v: the first edge that comes back to v
    # closes
    # the shortest cycle through v
    adj = make_graph(n, edges)
    dist, queue = {v: 0}, deque([v])
    while queue:
        u = queue.popleft()
        for w, _ in adj[u]:
            if w == v:
                return dist[u] + 1
            if w not in dist:
                dist[w] = dist[u] + 1
                queue.append(w)
    return -1


# FOLLOW-UP 1: How do you find the shortest cycle in the
# whole graph?
# Answer: Run the same BFS from every vertex and take the
# minimum: O(V * (V + E)). For undirected graphs, BFS from
```

```
# each node and treat any non-tree edge as closing a cycle
# of length dist[u] + dist[w] + 1 (the LeetCode 2608
# version).
# FOLLOW-UP 3: The shortest cycle anywhere in the graph
# (the girth).
# Answer: Run the same BFS from every node and take the
# smallest result: O(V * (V + E)). That is the standard
# way and fine for graphs up to a few thousand nodes.
def girth(n, edges):
    # the shortest cycle anywhere: BFS from every node
    sizes = [shortest_cycle_through(n, edges, v)
             for v in range(n)]
    found = [s for s in sizes if s > 0]
    return min(found) if found else -1


# FOLLOW-UP 2: What if edges have positive weights?
# Answer: Replace BFS with Dijkstra from v and close the
# cycle at the cheapest dist[u] + weight(u, v). That is
# O((V + E) log V) with a heap. With negative weights use
# Bellman-Ford and handle negative cycles separately.
def weighted_cycle_through(n, edges, v):
    # positive weights: Dijkstra, then close the cycle at
    # v
    adj = make_graph(n, edges)
    dist, heap, best = {v: 0}, [(0, v)], INF
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist.get(u, INF):
            continue
        for w, cost in adj[u]:
            if w == v:
                best = min(best, d + cost)
            elif d + cost < dist.get(w, INF):
                dist[w] = d + cost
                heapq.heappush(heap, (d + cost, w))
    return -1 if best == INF else best
```

```
if __name__ == "__main__":
    ring = [(0, 1), (1, 2), (2, 0)]
    assert shortest_cycle_through(3, ring, 0) == 3
    path = [(0, 1), (1, 2)]
    assert shortest_cycle_through(3, path, 0) == -1
    # a self loop is a cycle of length 1
```

```
    assert shortest_cycle_through(2, [(0, 0)], 0) == 1
    # a shorter cycle exists elsewhere: 1 <-> 2
    both = ring + [(1, 2), (2, 1)]
    assert girth(3, both) == 2
    assert girth(3, [(0, 1), (1, 2)]) == -1
    # weighted: the cheap way back is 0 -> 1 -> 0 (cost 3)
    weighted = [(0, 1, 1), (1, 0, 2),
                (0, 2, 1), (2, 0, 9)]
    assert weighted_cycle_through(3, weighted, 0) == 3
```

NOTES FROM CANDIDATES

Strictly directed; do not relax to undirected.
BFS layer 0 is v itself; the first re-discovery during expansion is the cycle length.
Common bug: marking v as visited immediately blocks the re-entry.

PREPARATION

Write the directed BFS-from-source version from memory in under 12 min.
Practice the "re-entry detection" pattern (don't mark v visited until after pop).